CP1 PHASE 3

Requirements, Concept Selection & System Design

Workbook Filling Guide

Item	Detail
Project	AI-Powered Real-Time Phishing URL Detection Using Machine Learning and Browser Extension
Prepared for	KL University Engineering Capstone Research Diary — Semester VII · CP1
Phase / weeks	Phase 3 · Weeks 6–8 · Gate: Review 1 @ Week 6 (feasibility gate)
Accreditation	PO2, PO3, PO5 · ABET SO1, SO2
Guide	Gogineni Krishna Chaithanya · CSE-HTE · KL University · 2026-2027 (from Review-2 PPT)
Source of truth	Project repository (code + docs/) as committed on 2026-09-29, cross-checked against docs/Phishing_Review2_Revised (1).pptx

How to read this guide. Every workbook page has the same parts: **A** what to write · **B** exact table content · **C** diagram (pages 36–38) · **D** how to draw / fill · **E** notes · **F** items you must supply. Anything marked **[NEEDS USER INPUT]** is not in the repository and must come from you. Anything marked **[CONFIRM]** is a reasonable reconstruction you should check before writing.

SECTION 1

Part A — Project Facts

All facts below are taken from the repository. The file that supports each fact is named so you can show it if a panel member asks.

Fact	Content
1. Project title	AI-Powered Real-Time Phishing URL Detection Using Machine Learning and Browser Extension *Source: PPT slides 1–2; docs/REVIEW_2_AND_VIVA_GUIDE.md §1*
2. Problem statement	Phishing websites imitate trusted sites to steal credentials. Users often cannot judge a link before opening it, and blocklists only know sites that were already reported. The team's earlier prototype needed 21 features, 15 of which required page HTML, WHOIS or ranking services, so it could not score an arbitrary URL automatically. *Source: PPT slide 3; guide §1; docs/dataset.md*
3. Objectives (numbered O1–O6 in PPT order)	**O1** Extract 17 features automatically from the URL string (same code for training and prediction). **O2** Train and evaluate ML models with leakage-controlled splits and honest metrics. **O3** Serve predictions through a secure Flask REST API. **O4** Store scan history in SQLite and show a dashboard. **O5** Provide a Chrome Manifest V3 extension that checks the current tab via the same API. **O6** Explain every result and state the model's measured limits. *Source: PPT slide 3; guide §1. The labels O1–O6 are assigned here in the documented order **[CONFIRM]**.*
4. Proposed solution	A URL-only ML risk signal: the server validates the URL, reduces it to its registrable domain (Public Suffix List), computes 17 lexical features, and a Gradient Boosting model returns P(phishing). Thresholds chosen on validation data give Safe / Suspicious / Phishing with reasons. Served by one Flask API to a web scanner + dashboard and a Chrome MV3 extension; scans stored in SQLite. The page is never downloaded.
5. Technology stack	**ML:** Python, scikit-learn 1.6.1 (pinned), pandas, numpy, joblib, tldextract 5.1.2 (offline PSL snapshot). **Backend:** Flask (REST/JSON). **Database:** SQLite (standard library), parameterised SQL, PRAGMA user_version migrations. **Frontend:** HTML5, Jinja2, CSS, vanilla JS (fetch), SVG charts; no framework/CDN. **Extension:** Chrome Manifest V3. **Testing:** pytest, headless Chrome via DevTools Protocol. **VCS:** Git / GitHub. *Source: requirements.txt; PPT slide 7*
6. Dataset	PhiUSIIL Phishing URL dataset (UCI id 967): 235,795 rows × 56 columns; label 0 = phishing, 1 = legitimate; 100,945 phishing / 134,850 legitimate. After removing 425 duplicate URLs and 8 invalid URLs: 235,362 rows (42.7% phishing) in 196,046 registrable-domain groups. Split 80/10/10 grouped by registrable domain, seed 42: train 187,523 · validation 24,784 · test 23,055. *Source: docs/dataset.md; docs/data/dataset_audit.json; PPT slide 8*
7. ML models	**Deployed:** GradientBoostingClassifier v2.0.0 (200 trees, depth 3, learning rate 0.1, seed 42). **Compared on validation (12 candidates):** Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, HistGradientBoosting, HistGradientBoosting (large) — each unweighted and class-balanced. **Not part of the system:** DistilBERT, XGBoost/MLP/SVM and ensemble scripts in the repo root are earlier research experiments on the leaky setup; they are not used by the application (README.md). *Source: models/model_metadata.json; guide §7*
8. Architecture summary	Three tiers in one Flask process: client tier (web scanner/dashboard, MV3 extension) → application tier (routes, validation, security controls, PredictionService, explanation engine, HistoryService) → ML & data tier (Predictor + feature extractor, model artefacts, SQLite). Model loaded once at start-up. Training pipeline runs offline. *Source: docs/architecture.md*
9. Key results (v2.0.0, held-out test, scored once)	Accuracy 0.7872 · Precision 0.8801 · **Recall 0.5675** · F1 0.6900 · ROC-AUC 0.8524 · FPR 0.0554. Confusion matrix: TN 12,691 · FP 744 · FN 4,161 · TP 5,459. Performance: 0.79 ms server-side model time (median); 4.17 ms per API request with history write; 862 req/s with 8 concurrent clients. *Never quote the ~99.97% figure — it came from a leaky feature and a biased split.*
10. Current implementation status	Completed (in repo): dataset audit, 17-feature extractor, training & selection, evaluation, Flask API, explanation engine, SQLite history, web scanner & dashboard, MV3 extension, automated tests (307 passing per docs/testing.md). Not done: WSGI/HTTPS/authentication deployment; Google Safe Browsing lookup exists in code but is disabled and unevaluated.

Source conflicts found and how they were resolved

The revised Review-2 PPT (untracked file in docs/) describes features that are **not** in the committed code. Because the code is the verifiable implementation, this guide uses the code. Confirm with your team whether the PPT features exist on another branch or machine.

Item	Committed code / docs (2026-09-29)	Revised Review-2 PPT	Used here
Extension behaviour	On-click popup, activeTab only, no service worker (extension/manifest.json)	Automatic scanning, service worker, badge, warning page, on/off switch	Code **[CONFIRM]**
API endpoints	9 routes: health, predict, model-info, features, history (list/get/delete/clear), stats	12 endpoints incl. 3 feedback endpoints	Code
Database	Schema v2, one table scan_history (database/models.py)	Schema v3 with user_feedback table	Code
Test count	307 tests (docs/testing.md)	443 tests	Code (not central to Phase 3)
Phishing share	42.7% after cleaning	42.8% (raw 100,945 / 235,795)	Both correct — different base
Team size	4 members (M1–M4) in docs	4 named members with IDs	4 members (prompt assumed 3)

Team (from PPT slide 1)

Student ID	Name	Role in docs
2300032709	Vasamsetti Madhukar	**[NEEDS USER INPUT]** (which of M1–M4?)
2300032696	Banka Ashish Kumar Yadav	**[NEEDS USER INPUT]**
2300032701	Akula Sai Ashritha	**[NEEDS USER INPUT]**
2300090189	Devapathni Varun	**[NEEDS USER INPUT]**

Roles defined in docs/REVIEW_2_AND_VIVA_GUIDE.md §19: **Member 1** ML / data · **Member 2** Backend / API / database · **Member 3** Frontend / dashboard · **Member 4** Chrome extension / integration / security / testing. The docs do not say which person holds which role, so this guide writes "Member 1 … Member 4". Replace with names once you confirm the mapping.

Page 31 — 3.1 Requirements Specification

A. WRITE THIS IN THE WORKBOOK (12 rows — fits the 10–12 row table)

Req. ID	Type	Requirement	Traces to
FR-01	Functional	System shall accept a URL from the web scanner or the extension via POST /api/predict and reject missing, malformed or non-http(s) URLs with a JSON error.	O3
FR-02	Functional	System shall reduce the URL to its registrable domain (PSL) and extract 17 lexical features in a fixed schema order, without fetching the page.	O1
FR-03	Functional	The same feature extractor shall be used for training and prediction (feature parity).	O1
FR-04	Functional	System shall de-duplicate the dataset and split it 80/10/10 grouped by registrable domain, so no domain is in two splits.	O2
FR-05	Functional	System shall train candidate classifiers and select model and threshold on validation data only; the test set is scored once.	O2
FR-06	Functional	System shall return P(phishing), a risk level (Safe / Suspicious / Phishing) and model confidence for each URL.	O2, O3
FR-07	Functional	System shall give reasons for each result and a disclaimer stating measured recall/FPR.	O6
FR-08	Functional	System shall store each scan (secrets redacted) in SQLite and show history and statistics on a dashboard.	O4
FR-09	Functional	Chrome MV3 extension shall send the active tab URL to the same API and show the result in a popup.	O5
NFR-01	Non-functional (performance)	Prediction shall be fast enough for interactive use; model loaded once at start-up, not per request.	O3
NFR-02	Non-functional (security)	API shall resist SSRF, XSS and SQL injection; apply rate limiting, exact-origin CORS and safe JSON errors.	O3
NFR-03	Non-functional (reliability / privacy)	Prediction shall still return if storage fails; only the URL is sent, no cookies/IP stored.	O3, O4, O5

D. HOW TO FILL PAGE 31

- Write one requirement per row exactly as above. Use "shall" — it marks a requirement, not a description.
- In the Type column, "Functional" or "Non-functional" is enough; the bracket (performance / security) is optional if space is short.
- Write O-codes only from the list O1–O6 in Part A. Write that list once at the top or bottom of the page so the examiner can read the trace column.
- If the table has only 10 rows, drop FR-03 (merge into FR-02) and NFR-03; move them to the Additional space page (Section 12).

E. NOTES — evidence for each requirement (do not write; use if asked)

ID	Implemented in	Verified by
FR-01	backend/routes/prediction.py, backend/utils/validation.py	tests/test_api.py (missing / malformed / unsupported scheme)
FR-02	ml/url_utils.py, ml/feature_extractor.py, ml/feature_schema.py	tests/test_features.py, tests/test_pipeline.py
FR-03	ml/dataset.py + ml/feature_extractor.py	test_training_and_prediction_features_are_identical; parity ≥ 99.99% (docs/data/feature_parity.json)
FR-04	ml/dataset.py (load_registrable_grouped_split)	test_split_reproduces_metadata_and_has_no_leakage
FR-05	ml/train_model.py, ml/model_utils.py	tests/test_model_selection.py
FR-06	ml/predictor.py; thresholds 0.5200 / 0.6443	test_risk_mapping_thresholds
FR-07	services/explanation_engine.py, services/prediction_service.py	tests/test_pipeline.py, tests/test_urls.py
FR-08	database/*, services/history_service.py, templates/dashboard.html	tests/test_database.py (38 tests)
FR-09	extension/manifest.json, extension/popup.js	tests/browser/test_extension_browser.py

ID	Implemented in	Verified by
NFR-01	backend/__init__.py (load once)	Measured 0.79 ms model / 4.17 ms request median (docs/data/performance.json)
NFR-02	backend/utils/security.py, rate_limit.py; database/repository.py	tests/test_security.py (58 tests)
NFR-03	prediction route + HistoryService; manifest permissions	test_secrets_in_scanned_url_never_reach_the_database

NFR-01 states no numeric target because the repository never set one in advance — only measured values exist. If your guide wants a number, write the target your team agreed **[NEEDS USER INPUT]** (e.g. a millisecond bound) and cite the measured 4.17 ms as evidence.

Page 32 — 3.2 Constraints & Standards to Comply With

A. WRITE THIS IN THE WORKBOOK

Constraint / Standard	Type	How it shapes the design
PhiUSIIL is the only labelled dataset; all legitimate URLs are bare homepages	Constraint (data)	Model scores the registrable domain only; metrics reported as a risk signal, not a guarantee.
Dataset column URLSimilarityIndex leaks the label	Constraint (data)	Excluded; only features recomputable from a URL string are used (17 of 22).
Server must not fetch submitted URLs (SSRF, malicious pages)	Design constraint (security)	URL-only lexical features; no HTML, WHOIS or DNS lookups.
Interactive, near-real-time response in the browser	Constraint (performance)	Lightweight Gradient Boosting; model loaded once; no per-request I/O except SQLite.
Student laptop (CPU only) for training and serving	Constraint (resources)	Classical ML trainable in seconds; transformer (DistilBERT) not adopted.
scikit-learn version lock (old pickle fails on ≥ 1.4)	Constraint (compatibility)	Pinned scikit-learn 1.6.1; feature names/order checked when the model loads.
Chrome Manifest V3 extension platform	Standard (platform)	manifest_version 3; minimal permissions: activeTab + API origin; CSP for extension pages.
HTTP/REST + JSON conventions	Standard (de facto)	Correct methods and status codes (400/413/415/422/429/500/503); one JSON error shape.
Public Suffix List (via tldextract, offline snapshot)	Standard (reference data)	Defines the registrable domain uniformly; not a whitelist.
Web threats: XSS, SQL injection, SSRF, CORS abuse	Guideline (secure coding)	textContent-only rendering, CSP, parameterised SQL, exact-origin CORS, rate limit.
User privacy of browsing URLs	Ethical / privacy consideration	Only the URL is sent; secrets redacted before storage; no IP/cookies stored; no third-party lookup by default.
CP1 timeline (Weeks 6–8, Review 1 gate) and 4-member team	Constraint (academic)	Local single-user deployment; HTTPS, WSGI and authentication deferred.

D. HOW TO FILL PAGE 32

- Write the Type column exactly — it shows the examiner you distinguish a **constraint** (a limit you must live with), a **standard** (a specification you conform to), a **guideline** (good practice you follow without a formal claim) and a **design consideration**.
- If space is short, keep rows 1, 2, 3, 4, 7, 10, 11 — they matter most for the design.

E. NOTES — what NOT to claim

- The repository does not reference OWASP, IEEE 830 / ISO/IEC/IEEE 29148, ISO 27001 or any other formal standard. Do not write "complies with" any of them. If asked, say: "we followed common secure-coding practice for these threat classes; no formal compliance was assessed".
- MV3 is a real conformance requirement: Chrome will not load the extension otherwise (tests/test_client_code.py::test_manifest_is_mv3_with_minimal_permissions).
- The "student laptop" constraint is supported by the comment in train_distilbert_clean.py ("For a MacBook Air, start with these conservative values") and the macOS benchmark in docs/testing.md.

Page 33 — 3.3 Candidate Solutions

Each concept is a genuine alternative for the problem in Part A. Concepts A, B and D each relate to something that exists in the repository's history: blocklist lookup (disabled Safe Browsing code), the earlier HTML/WHOIS prototype, and the DistilBERT experiment scripts.

A. WRITE THIS IN THE WORKBOOK

Concept	Description	Pros	Cons
A. Blocklist / rule-based check	Look the URL up in a reputation blocklist (e.g. Google Safe Browsing) and/or apply hand-written URL rules.	Very fast; simple; easy to explain; near-zero false alarms on listed sites.	Misses new, unreported phishing; lists need constant updates; URL sent to third party.
B. Page-content + URL ML (earlier prototype)	Fetch the page and use HTML, WHOIS, DNS and traffic features (21 features) in an ML classifier.	Richer signals; can catch look-alike pages with ordinary URLs.	Server must fetch hostile pages (SSRF); slow; external services retired/paid; old model not reusable.
C. URL-only lexical features + classical ML (Gradient Boosting) behind a REST API	Compute 17 features from the URL string (registrable domain); tree-ensemble classifier returns P(phishing); served to web UI and MV3 extension.	Scores never-seen URLs; ~1 ms inference; no page fetch; explainable features; CPU-only.	URL string has limited information — recall ceiling; sensitive to dataset bias.
D. Transformer on raw URL text (DistilBERT)	Tokenise the full URL and fine-tune DistilBERT as a sequence classifier.	Learns character/token patterns automatically; no manual features.	Heavy for CPU serving; low explainability; full-URL models learn the dataset's homepage shortcut.

Recommended concept: C — URL-only lexical features + classical ML (Gradient Boosting) behind a Flask REST API, consumed by a web dashboard and a Chrome MV3 extension. (Justified on page 34.)

D. HOW TO FILL PAGE 33

- If the workbook table has 3 rows, write A, B and C; write D under the table or on the Additional space page.
- Underline or star Concept C and write "Recommended" beside it.

E. NOTES — evidence behind each concept

- **A:** services/threat_intelligence.py supports Google Safe Browsing but is disabled and never evaluated. docs/model_evaluation.md shows a simple rule scores 0.996 F1 on the original split — only because of dataset bias, which is why rules were rejected as a detector.
- **B:** docs/dataset.md — the earlier notebook model used 21 features; 15 need page HTML or external services (Alexa traffic is retired, PageRank is not published). The pickle cannot load with scikit-learn ≥ 1.4.
- **C:** the implemented system (docs/architecture.md).
- **D:** train_distilbert*.py exist but were run on the earlier leaky setup; a fair DistilBERT comparison on the leakage-controlled split is listed as future work (guide §18). Do **not** claim DistilBERT results.

Page 34 — 3.4 Decision / Trade-off Matrix

Scale: 1 = Poor · 2 = Fair · 3 = Good · 4 = Very good · 5 = Excellent. Weighted total = Σ (weight × score) ÷ 100 (maximum 5.00). Scores are the team's engineering judgement at design time; they are not measured results.

A. WRITE THIS IN THE WORKBOOK

Criterion	Weight	A Blocklist / rules	B Page + URL ML	C URL-only GB (selected)	D DistilBERT
Detection of new / unseen phishing URLs	20%	1 (0.20)	4 (0.80)	3 (0.60)	3 (0.60)
Real-time latency in the browser	15%	5 (0.75)	2 (0.30)	5 (0.75)	2 (0.30)
Security & privacy (no page fetch, SSRF-safe)	15%	3 (0.45)	1 (0.15)	5 (0.75)	5 (0.75)
False-positive control	10%	4 (0.40)	3 (0.30)	3 (0.30)	2 (0.20)
Explainability of the result	10%	5 (0.50)	3 (0.30)	4 (0.40)	2 (0.20)
Feasibility within CP1 (complexity, CPU only)	10%	5 (0.50)	2 (0.20)	4 (0.40)	2 (0.20)
Integration with API + extension	10%	4 (0.40)	2 (0.20)	5 (0.50)	3 (0.30)
Maintainability & reproducibility	10%	2 (0.20)	2 (0.20)	4 (0.40)	3 (0.30)
Weighted total (out of 5)	**100%**	**3.40**	**2.45**	**4.10**	**2.85**
Rank		**2**	**4**	**1**	**3**

Cell format: score (weighted points). Example — C, detection: 20 × 3 ÷ 100 = 0.60.

Selected concept: C — URL-only lexical features + Gradient Boosting behind a Flask REST API (weighted total 4.10 / 5).

Selected concept & justification (write this under the matrix)

Concept C scores highest (4.10 vs 3.40 for A). It is the only option that scores never-seen URLs while keeping inference near-instant and never fetching the page, which removes SSRF risk and protects user privacy. Its 17 named features make results explainable, it trains on a CPU, and one REST API serves both the web dashboard and the MV3 extension. Accepted trade-off: a URL string carries limited information, so recall is capped; the system is presented as a risk signal, with limits stated.

D. HOW TO FILL PAGE 34

- The workbook has columns Concept A, B, C. Write A, B, C as above. Add D as a fourth column if there is room; otherwise add one line under the table: "D (DistilBERT) = 2.85 — rejected: CPU cost, low explainability."
- Write only the score in each cell if space is tight; write the weighted totals in the last row.
- Shade or box the column of the selected concept (C).

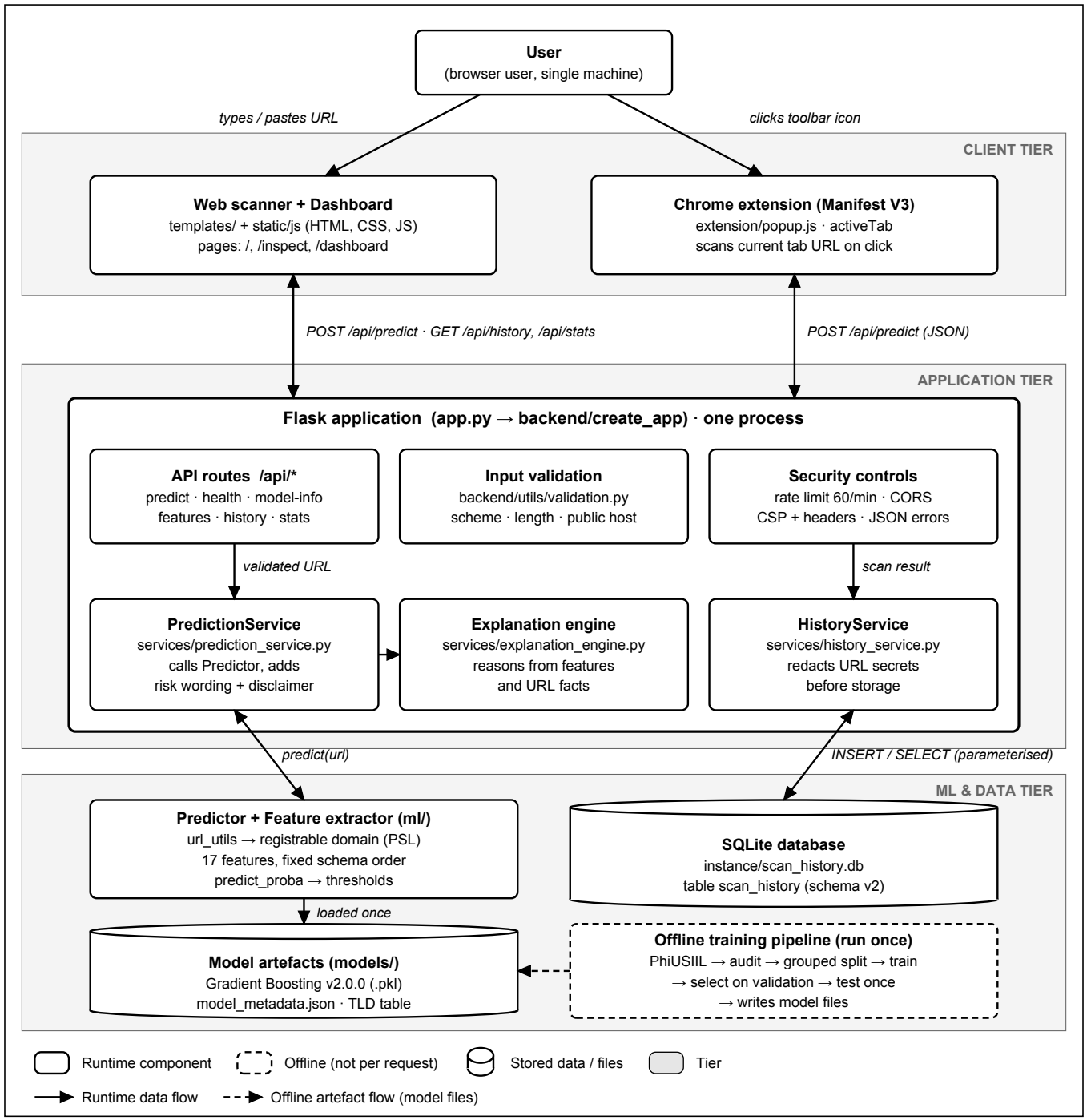
E. NOTES — how later evidence supports the choice (for viva only)

- Measured after implementation: 0.79 ms model time; 12 classical candidates compared and Gradient Boosting gave the only significant recall gain at 5% FPR (bootstrap CI [+0.1317, +0.1482]).
- The 'False-positive control = 3' score for C reflects the documented subdomain problem found later and fixed with the registrable-domain view (subdomain FPR 17.8% → 1.2% on validation).
- These numbers did not exist in Weeks 6–8; do not write them as design-time facts on page 34.

Page 36 — 3.5(a) High-Level Architecture Diagram

C. DIAGRAM

Figure 3.1 — High-level architecture of the phishing URL detection system



A. WRITE THIS IN THE WORKBOOK

Title: Figure 3.1 — High-level architecture of the phishing URL detection system. Draw the diagram, then the legend. Below it write: "One Flask process serves the web pages and the REST API and is the single source of predictions; the web UI and the extension contain no model logic."

D. HAND-DRAWING VERSION (simplified)

```
                        [ User ]
                 types URL /   \ clicks icon
                           v     v
CLIENT    [ Web scanner + Dashboard ]    [ Chrome MV3 extension ]
                 |  ^  POST /api/predict (JSON)  |  ^
                 v  |  JSON response             v  |
APP     +---------------- Flask REST API ------------------+
        | [Routes] -> [Validation] -> [Security: CORS/CSP/ |
        |                              rate limit]         |
        | [PredictionService] [Explanation] [HistoryService]|
        +-------|------------------------------|-----------+
                v predict(url)                 v redacted record
ML/DATA [ Predictor + Feature extractor ]   ( SQLite scan_history )
                | loaded once
        ( Model artefacts: GB v2.0.0 .pkl ) <- - - [ Offline training ]

Legend: [ ] component   ( ) stored data   ---> runtime flow   - - -> offline
```

- Use a pencil and ruler. Draw three horizontal bands first (Client / Application / ML & Data) and label them on the left.
- Draw the Flask box large — it contains six small boxes. Rounded corners are not needed.
- Use a solid arrow for runtime requests, a dashed arrow for the offline training output, and a cylinder for SQLite and the model files.
- Put the legend in the bottom-right corner inside a small rectangle.

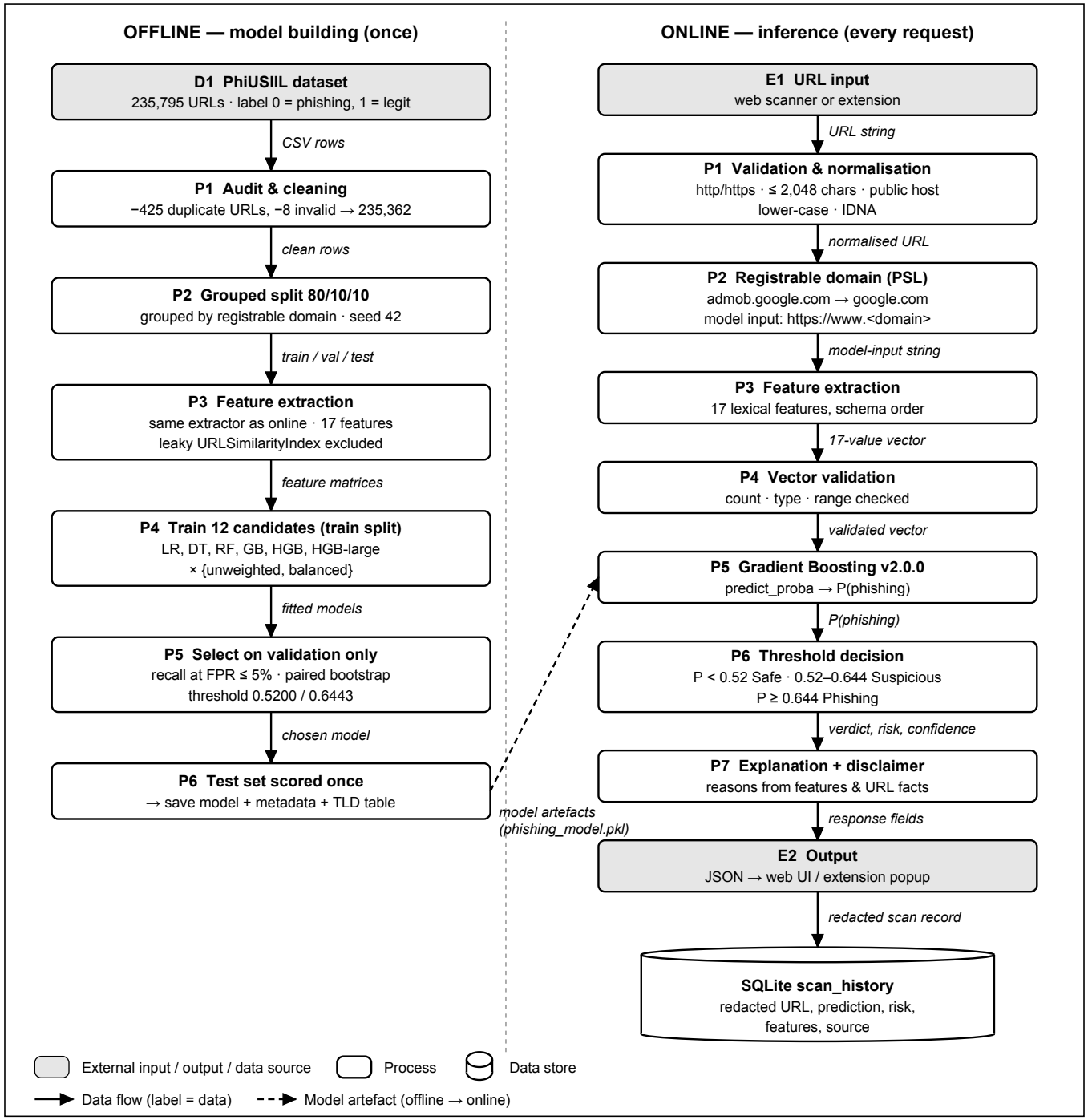
E. WHAT EACH BOX MEANS

Box	Meaning	Code
User	Person checking a link on their own machine (single-user, no login)	—
Web scanner + Dashboard	Pages to paste a URL, see the verdict, and view history/statistics	templates/, static/js/
Chrome MV3 extension	Popup that sends the current tab URL to the API when the icon is clicked	extension/
API routes	HTTP endpoints under /api (9 routes)	backend/routes/
Input validation	JSON shape, URL length ≤ 2,048, http/https only, public host	backend/utils/validation.py
Security controls	Rate limit, exact-origin CORS, CSP and hardening headers, uniform JSON errors	backend/utils/
PredictionService	Calls the predictor, adds risk wording, disclaimer, model version, time	services/prediction_service.py
Explanation engine	Turns features and URL facts into human-readable reasons	services/explanation_engine.py
HistoryService	Redacts secrets, writes the scan to SQLite; failure never blocks prediction	services/history_service.py
Predictor + Feature extractor	Normalise URL → registrable domain → 17 features → P(phishing) → thresholds	ml/
Model artefacts	Trained Gradient Boosting v2.0.0, metadata, TLD prior table	models/
SQLite	Local file database of past scans	database/, instance/
Offline training pipeline	Audit, split, train, select, test — run by developers, not per request	ml/train_model.py, scripts/

Page 37 — 3.5(b) Data-Flow / Block Diagram

C. DIAGRAM

Figure 3.2 — Data flow: offline model building and online inference



A. WRITE THIS IN THE WORKBOOK

Title: Figure 3.2 — Data flow: offline model building and online inference. Below the diagram write: "The same feature extractor is used in both paths, so training-time and prediction-time features are identical. The deployed system uses one model (Gradient Boosting) — there is no ensemble and no DistilBERT stage."

D. HAND-DRAWING VERSION (simplified)

```
OFFLINE (once)                          ONLINE (each request)
(PhiUSIIL 235,795 URLs)                 [URL from web UI / extension]
  | CSV rows                              | URL string
  v                                       v
[Audit: -425 dup, -8 invalid]           [Validate + normalise]
  v                                       v normalised URL
[Grouped split 80/10/10]                [Registrable domain (PSL)]
  v                                       v https://www.<domain>
[Extract 17 features] <-same code->     [Extract 17 features]
  v                                       v 17-value vector
[Train 12 candidates]                   [Validate vector]
  v                                       v
[Select on validation, thr 0.52]        [Gradient Boosting -> P(phishing)]
  v                                       v
[Test once -> model.pkl] - - - - - - -> (model)
                                        [Thresholds: Safe/Suspicious/Phishing]
                                          v
                                        [Explanation + disclaimer]
                                          v
                                        [JSON output] --> (SQLite history)
```

- Draw two vertical columns. Left = offline (training), right = online (inference). Separate them with a dotted line.
- Write the data name on every arrow (e.g. "17-value vector") — that is what makes it a data-flow diagram, not a flowchart.
- Shade the external input/output boxes (dataset, URL input, output); draw processes as plain rectangles and SQLite as a cylinder.
- Draw the dashed "model artefacts" arrow from the last offline box to the Gradient Boosting box.

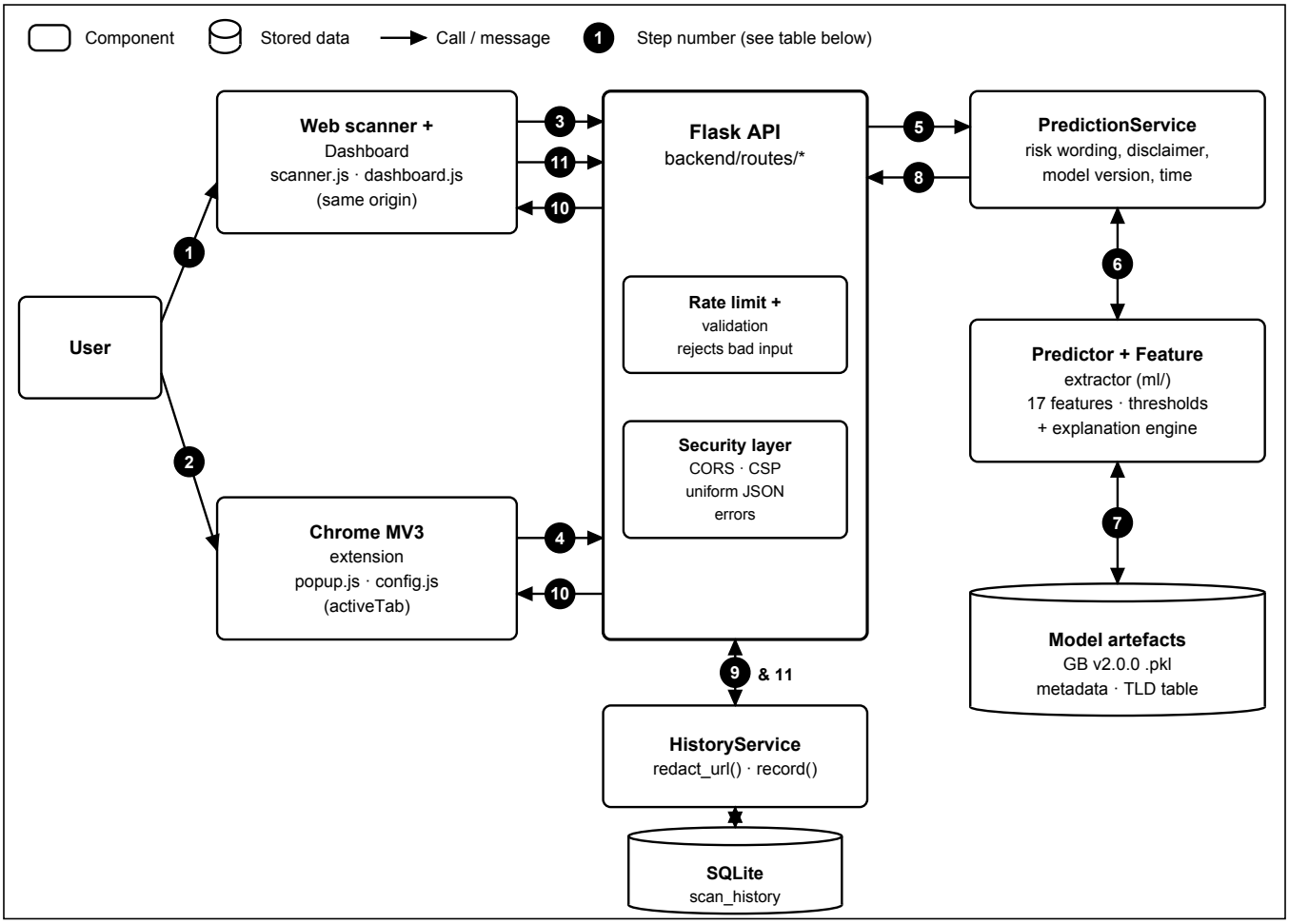
E. NOTES

- Threshold values: 0.5200 = lowest threshold with validation FPR ≤ 5%; 0.6443 = high-confidence cut (models/model_metadata.json).
- No tokenizer/embedding stage exists in the implemented system. DistilBERT appears only as rejected Concept D.

Page 38 — 3.5(c) Component-Interaction Diagram

C. DIAGRAM

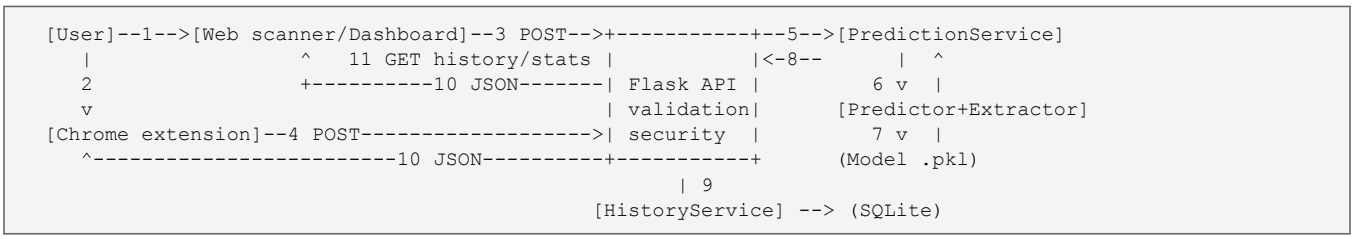
Figure 3.3 — Component interaction for one URL check



A. WRITE THIS IN THE WORKBOOK — interaction table (write under the diagram)

#	From → To	Interaction / data
1	User → Web scanner	Enters or pastes a URL
2	User → Extension	Clicks toolbar icon; activeTab grants the current tab URL
3	Web scanner → Flask API	HTTP POST /api/predict {url, source: "web"} (same origin)
4	Extension → Flask API	HTTP POST /api/predict {url, source: "extension"} (host_permissions)
5	Flask API → PredictionService	Rate-limited, validated URL
6	PredictionService ↔ Predictor	predict(url) → features, P(phishing), verdict, explanation
7	Predictor ↔ Model artefacts	17-feature vector → predict_proba (model loaded once)
8	PredictionService → Flask API	Prediction result + risk wording + disclaimer
9	Flask API → HistoryService → SQLite	Redacted scan record → history id (failure does not block)
10	Flask API → Web / Extension	JSON response (200) or JSON error {code, message}
11	Dashboard → Flask API → SQLite	GET /api/history, /api/stats → table and charts

D. HAND-DRAWING VERSION (simplified)



- Three columns: clients on the left, Flask API in the middle, ML on the right; HistoryService and SQLite under the API.
- Number each arrow and write the matching row in the interaction table — this keeps the diagram uncluttered.
- Two-headed arrows (6, 7) mean call and return.

E. COMPONENTS & CONNECTIONS — explanation

- **Clients hold no model.** Both call the same endpoint, so web and extension results are identical (tested: test_https_page_matches_web_app_prediction).
- **Validation before prediction:** bad input returns 400 (invalid URL / scheme / JSON), 413 (body > 16 KB), 415, or 429 (rate limit) and never reaches the model.
- **Fail-safe paths:** model missing → 503 and /api/health shows model_loaded: false; SQLite failure → prediction still returned with history.saved = false.
- **No outbound network calls:** no component fetches the submitted URL (test_scanning_never_opens_network_connections).

Page 39 — 3.6 Detailed Design of CP1 Objectives (O1–O2)

O1 — Extract 17 features automatically from the URL string (same code for training and prediction).
O2 — Train and evaluate ML models with leakage-controlled splits and honest metrics.
*These are the first two objectives listed in the PPT (slide 3) and the review guide (§1). The repository does not label them "CP1" — **[CONFIRM]** that your CP1 objectives are O1 and O2 as numbered here.*

A. WRITE THIS IN THE WORKBOOK

Component	Description	Inputs	Outputs	Dependencies
O1 · URL utilities	Normalise URL; split host into subdomain + registrable domain (PSL)	Raw URL string	Normalised URL, host, registrable domain	urllib, ipaddress, IDNA, tldextract
O1 · Feature schema	Single ordered list of 17 features; excluded features; vector checks	Feature vector	Validated vector or error	Python (no external)
O1 · Feature extractor	Builds https://www.<domain> and computes the 17 features	URL string	17-value vector (schema order)	URL utilities, schema, TLD prior table
O1 · Parity audit	Checks recomputed features equal the dataset's stored columns	PhiUSIIL rows (URL + stored features)	feature_parity.json (≥ 99.99%)	Extractor, pandas
O2 · Dataset audit	Finds duplicates, invalid URLs, label conflicts, split overlap	PhiUSIIL CSVs	dataset_audit.json	pandas, URL utilities
O2 · Grouped split loader	De-duplicate; 80/10/10 split grouped by registrable domain, stratified, seed 42	Clean rows	Train / validation / test + SHA-256 fingerprints	pandas, scikit-learn
O2 · Training & selection	Fit 12 candidates; pick model + threshold at validation FPR ≤ 5% (paired bootstrap)	Train + validation feature matrices	Selected model, thresholds 0.52 / 0.644	scikit-learn 1.6.1, numpy
O2 · Evaluation & error analysis	Score test set once; confusion matrix, error and calibration analysis	Test set, saved model	Metrics, JSON reports	scikit-learn metrics
O2 · Model artefacts + Predictor	Save/load model with metadata; check feature order on load	model .pkl, metadata, TLD table	P(phishing), risk level	joblib, feature schema

D. HOW TO FILL PAGE 39

- Write the O1/O2 statements at the top of the page, then the table.
- The "O1 ·" / "O2 ·" prefix in the Component column shows the mapping without needing an extra column.
- If the table has fewer rows, merge "Feature schema" into "Feature extractor" and "Dataset audit" into "Grouped split loader".

E. NOTES — file names (if the examiner asks "where is it?")

Component	File(s)
URL utilities	ml/url_utils.py
Feature schema	ml/feature_schema.py
Feature extractor	ml/feature_extractor.py; models/tld_legitimate_prob.json
Parity audit	scripts/feature_parity_audit.py → docs/data/feature_parity.json
Dataset audit	scripts/dataset_audit.py → docs/data/dataset_audit.json
Grouped split loader	ml/dataset.py
Training & selection	ml/train_model.py, ml/model_utils.py
Evaluation & error analysis	ml/evaluate_model.py, ml/error_analysis.py, ml/view_experiment.py, scripts/calibration_report.py
Model artefacts + Predictor	models/phishing_model.pkl, models/model_metadata.json, ml/predictor.py

Page 40 — 3.7 Risk Register

A. WRITE THIS IN THE WORKBOOK

Risk ID	Description	Likelihood	Impact	Mitigation
R-01	Dataset bias: legitimate URLs are only homepages → misleading metrics	High	High	Score registrable domain only; bias probe; report limits honestly
R-02	Data leakage (duplicate URLs/domains across splits, leaky column)	High	High	De-duplicate; split grouped by registrable domain; drop URLSimilarityIndex; test once
R-03	High false negatives — URL alone lacks information	High	High	Threshold at validation FPR ≤ 5%; "risk signal" wording; plan reputation/domain-age signals
R-04	False positives on legitimate subdomains (e.g. admob.google.com)	Medium	High	PSL registrable-domain input; subdomain-FPR check in model selection; no whitelist
R-05	Feature mismatch or wrong order between training and prediction	Medium	High	One schema; vector validation; model refuses to load if order differs; parity audit
R-06	Overfitting / selecting the model on test data	Medium	High	Select on validation only; depth-3 trees; paired bootstrap; test scored once
R-07	Library incompatibility (old pickle fails on new scikit-learn)	Medium	Medium	Pin scikit-learn 1.6.1; retrain; record environment in metadata
R-08	Hostile input: SSRF, XSS, SQL injection	Medium	High	Never fetch URLs; textContent + CSP; parameterised SQL; security tests
R-09	API abuse / oversized requests	Medium	Medium	60 predictions/min per client; 16 KB body; 2,048-char URL limit
R-10	Model file missing or corrupt at start-up	Low	High	Health reports model_loaded; predict returns 503; app still starts
R-11	Privacy of users' browsing URLs	Medium	Medium	Send URL only; activeTab; redact secrets; no IP/cookies; no third-party lookup by default
R-12	Model drift as phishing tactics change	Medium	Medium	Versioned models + metadata; reproducible retraining script; drift monitoring as future work

D. HOW TO FILL PAGE 40

- Write L/M/H if the Likelihood/Impact columns are narrow.
- If the page fits only 8 rows, keep R-01, R-02, R-03, R-04, R-05, R-06, R-08, R-11.

E. NOTES — which risks actually occurred (good viva material)

- **R-02 occurred:** 63 duplicate test URLs, 2,025 shared hosts, 3,317 sibling-subdomain rows were found and removed by grouping (guide §5).
- **R-04 occurred:** model v1.2.0 gave admob.google.com P = 0.98; fixed in v2.0.0 at the cost of recall (docs/false_positive_investigation.md).
- **R-07 occurred:** the earlier Phishing_model.pkl fails with ModuleNotFoundError on scikit-learn ≥ 1.4.
- Excluded as not relevant: DistilBERT compute risk (not adopted), webpage-feature extraction failure (no page fetching).

Page 41 — 3.8 Project Plan / Work Breakdown

Tasks are the work documented in the repository. The repository has no week-by-week log and all code was committed on 2026-09-03 / 2026-09-29, so the week allocation below is a plan reconstruction **[CONFIRM]**. Roles follow guide §19 (Member 1 ML/data · 2 Backend/DB · 3 Frontend · 4 Extension/security/testing).

A. WRITE THIS IN THE WORKBOOK

#	Phase / Task	Owner(s)	Start	End	Dependencies
T1	Finalise problem statement, objectives O1–O6, scope	All	Week 6	Week 6	Phase 2 literature review
T2	Dataset feasibility audit (duplicates, invalid URLs, leakage)	Member 1	Week 6	Week 6	PhiUSIIL download
T3	Assess earlier prototype for reuse (21 features, old pickle)	Member 1, 2	Week 6	Week 6	—
T4	Requirements specification (FR / NFR) + constraints	All	Week 6	Week 7	T1, T2
T5	Review 1 feasibility presentation	All	Week 6	Week 6	T1–T3
T6	Candidate concepts + weighted trade-off matrix	All	Week 7	Week 7	T4
T7	Feature schema design (17 features, exclusions)	Member 1	Week 7	Week 7	T2, T6
T8	Leakage-controlled split design	Member 1	Week 7	Week 8	T2
T9	Architecture + API contract (endpoints, JSON, errors)	Member 2	Week 7	Week 8	T6
T10	SQLite schema for scan history	Member 2	Week 7	Week 8	T9
T11	UI design: scanner, result view, dashboard	Member 3	Week 7	Week 8	T9
T12	Extension design (MV3, permissions) + threat list	Member 4	Week 7	Week 8	T9
T13	Diagrams 3.5(a)–(c), detailed design O1–O2	All	Week 8	Week 8	T7–T12
T14	Risk register + Phase 3 documentation	Member 4, all	Week 8	Week 8	T13
T15	Start O1 implementation: URL utilities + extractor	Member 1	Week 8	Phase 4	T7

D. HOW TO FILL PAGE 41

- Write "Week 6/7/8" — do not write calendar dates unless you have them **[NEEDS USER INPUT]**.
- Replace "Member 1–4" with names only after confirming the role mapping **[NEEDS USER INPUT]**.
- If the table is short, merge T2+T3, T10+T11, and T13+T14.

E. NOTES

- Git history shows a contribution by student 2300032701 ("Update create_clean_split.py", 2026-09-03) and the main code commits by unispend001 / Devapathni Varun (2026-09-29). Commit dates do not record when each task was done; use them only as evidence that the work exists, not as task dates.

SECTION 12

Additional Space — Requirements & Design (optional continuation)

Write any of these if the workbook's "Additional space — Requirements & design" page needs content, or if rows did not fit on pages 31–41.

A. WRITE THIS IN THE WORKBOOK — additional requirements

Req. ID	Type	Requirement	Traces to
NFR-04	Non-functional (maintainability)	Model files shall carry metadata (version, features, thresholds, metrics); training reproducible with seed 42.	O2
NFR-05	Non-functional (usability)	Results shall use honest wording ("low-risk prediction", "model confidence"); UI usable on mobile width.	O6, O4
NFR-06	Non-functional (compatibility)	System shall run with pinned scikit-learn 1.6.1 and check the model's feature order on load.	O2
NFR-07	Non-functional (scalability)	CP1 targets local single-user deployment; WSGI multi-worker deployment is planned, not built.	O3

A. Key design decisions

Decision	Reason
URL-only, never fetch the page	Privacy, speed, removes SSRF risk
Gradient Boosting (tree ensemble) on 17 tabular features	Strong on low-dimensional tabular data; no scaling needed; CPU-fast
Model input = registrable domain (PSL)	Stops legitimate subdomains being flagged; not a whitelist
Threshold chosen at validation FPR ≤ 5%	Caps false alarms so users keep trusting warnings
One API for web + extension	One source of truth; identical results on both clients
SQLite for history	Local, zero-configuration, standard library, single user
activeTab-only extension	Minimal permission; URL read only when the user clicks

A. Assumptions

- Users run the system locally; no authentication is required in CP1.
- PhiUSIIL labels are correct (0 conflicting labels found in the audit).
- The bundled Public Suffix List snapshot is current enough for registrable-domain splitting.
- The output is a risk signal to support a user's decision, not an automatic block.

A. Model / design considerations

- Features 12–14 and 17 are constant in the registrable view; kept so the schema stays stable.
- Probability is uncalibrated (validation ECE 0.061) — shown as "model confidence", not certainty.
- Known trade-off: fixing subdomain false positives reduced recall (0.68 → 0.57 on the same test set).

SECTION 13

Week 06 — Weekly Progress Log

SEMESTER VII · CP1 · WEEKLY PROGRESS LOG · Phase 3 — Requirements, Concept Selection & Design

Important: the repository contains no dated weekly log, meeting notes or guide comments. The content below is built only from work that the repository documents, allocated to weeks in a logical order. Confirm each line with your team before writing **[CONFIRM]**. Progress percentages are suggestions.

A. WRITE THIS IN THE WORKBOOK

Field	Content
Week No.	06
Phase	Phase 3 — Requirements, Concept Selection & Design
Dates (from – to)	**[NEEDS USER INPUT]**
Planned agenda / focus	Review 1 (feasibility gate). Finalise problem statement and objectives O1–O6; assess dataset and earlier prototype; draft requirements.

Member-wise work done this week

Member	Task	Contribution	Progress
Member 1 (ML / data)	Dataset feasibility audit	Audited PhiUSIIL: 235,795 rows, 425 duplicate URLs, 8 invalid URLs, label-leaking column identified	60%
Member 2 (Backend / DB)	Prototype reuse assessment; API needs	Found earlier pickle will not load on scikit-learn ≥ 1.4; listed API functions needed (predict, health, history)	50%
Member 3 (Frontend)	UI requirements	Listed scanner and result-display requirements; reviewed what users must see (verdict, risk, reasons)	40%
Member 4 (Extension / security)	Platform + security requirements	Studied Chrome MV3 and activeTab; listed threat classes (SSRF, XSS, SQLi) as NFRs	40%

Field	Content
Actionable items completed	• Problem statement and objectives O1–O6 finalised • Dataset feasibility confirmed; leakage and duplicate issues recorded • Draft FR/NFR list prepared • Review 1 feasibility presentation delivered **[CONFIRM]**
Challenges faced & how addressed	**Earlier prototype not reusable** — 15 of its 21 features need page HTML, WHOIS or ranking services → decided to design URL-only features. **Dataset leakage** — URLSimilarityIndex equals 100 for every legitimate row → marked for exclusion.
Evidence produced this week	• docs/dataset.md (dataset facts, prototype assessment) • scripts/dataset_audit.py → docs/data/dataset_audit.json • Review 1 slides **[NEEDS USER INPUT]** (no Review-1 file in repository)
GUIDE FEEDBACK / COMMENTS	[Guide to complete after review]
Action plan for next week	• Generate candidate concepts and build trade-off matrix • Design the 17-feature schema and leakage-controlled split • Draft architecture and API contract
Guide Signature & Date	[Guide signs]
Team Lead Signature & Date	[Team lead signs] — team lead **[NEEDS USER INPUT]**

E. SUGGESTED POINTS TO RAISE WITH THE GUIDE (not guide feedback — do not write as feedback)

- Is a URL-only approach acceptable for the project scope?
- Is PhiUSIIL alone sufficient, given its homepage-only legitimate class?

SECTION 14

Week 07 — Weekly Progress Log

SEMESTER VII · CP1 · WEEKLY PROGRESS LOG · Phase 3 — Requirements, Concept Selection & Design

Important: the repository contains no dated weekly log, meeting notes or guide comments. The content below is built only from work that the repository documents, allocated to weeks in a logical order. Confirm each line with your team before writing **[CONFIRM]**. Progress percentages are suggestions.

A. WRITE THIS IN THE WORKBOOK

Field	Content
Week No.	07
Phase	Phase 3 — Requirements, Concept Selection & Design
Dates (from – to)	**[NEEDS USER INPUT]**
Planned agenda / focus	Concept generation and selection; feature and split design; first architecture and interface designs.

Member-wise work done this week

Member	Task	Contribution	Progress
Member 1 (ML / data)	Feature schema + split design	Defined 17 URL-only features and 5 excluded features; designed 80/10/10 grouped, stratified split (seed 42)	65%
Member 2 (Backend / DB)	Architecture + API contract	Designed single Flask API as the source of predictions; endpoint list, JSON request/response and error shape	60%
Member 3 (Frontend)	UI design	Designed scanner page, result card (verdict, risk, confidence, reasons) and dashboard layout	55%
Member 4 (Extension / security)	Extension + security design	Designed MV3 popup flow (activeTab → API → result) and security controls list	55%

Field	Content
Actionable items completed	• Four candidate concepts documented (A–D) • Weighted trade-off matrix completed; Concept C selected • Feature schema (17 features, fixed order) defined • High-level architecture drafted (Figure 3.1)
Challenges faced & how addressed	**Choosing between transformer and classical ML** — DistilBERT is heavy on a CPU laptop and hard to explain → classical ML on tabular features selected; DistilBERT kept as a later comparison. **Random split leaks duplicates/hosts** → designed a grouped split.
Evidence produced this week	• ml/feature_schema.py (17 FEATURE_NAMES, EXCLUDED_FEATURES) • docs/architecture.md (architecture, request flow) • docs/api.md (endpoint contract) • Phase 3 workbook pages 33–34
GUIDE FEEDBACK / COMMENTS	[Guide to complete after review]
Action plan for next week	• Finalise requirements and diagrams 3.5(a)–(c) • Complete detailed design O1–O2, risk register, WBS • Start the URL utilities and feature extractor
Guide Signature & Date	[Guide signs]
Team Lead Signature & Date	[Team lead signs] — team lead **[NEEDS USER INPUT]**

E. SUGGESTED POINTS TO RAISE WITH THE GUIDE (not guide feedback — do not write as feedback)

- Is the weighting in the trade-off matrix acceptable?
- Should a deep-learning comparison be in CP1 or CP2 scope?

SECTION 15

Week 08 — Weekly Progress Log

SEMESTER VII · CP1 · WEEKLY PROGRESS LOG · Phase 3 — Requirements, Concept Selection & Design

Important: the repository contains no dated weekly log, meeting notes or guide comments. The content below is built only from work that the repository documents, allocated to weeks in a logical order. Confirm each line with your team before writing **[CONFIRM]**. Progress percentages are suggestions.

A. WRITE THIS IN THE WORKBOOK

Field	Content
Week No.	08
Phase	Phase 3 — Requirements, Concept Selection & Design
Dates (from – to)	**[NEEDS USER INPUT]**
Planned agenda / focus	Close Phase 3: finalise requirements, selected concept, architecture, detailed design, risks and plan; begin O1 implementation.

Member-wise work done this week

Member	Task	Contribution	Progress
Member 1 (ML / data)	Detailed design O1–O2; start extractor	Specified inputs/outputs of URL utilities, extractor, split loader, training and evaluation; began extractor + parity check	75%
Member 2 (Backend / DB)	Data-flow + DB design	Finalised request flow (validate → predict → record → JSON); designed scan_history table and redaction rule	70%
Member 3 (Frontend)	Interaction design	Mapped UI calls to API (POST predict, GET history/stats); defined error states	65%
Member 4 (Extension / security)	Risk register + diagrams	Prepared risk register (R-01–R-12) and component-interaction diagram	70%

Field	Content
Actionable items completed	• Requirements specification finalised (FR-01–FR-09, NFR-01–NFR-07) • Selected concept C confirmed • Diagrams 3.5(a), (b), (c) finalised • Detailed design O1–O2, risk register and WBS completed • Phase 3 workbook section completed
Challenges faced & how addressed	**Feature order mismatch risk** — model reads features by position → single schema plus load-time order check designed. **Threshold policy** — needed a rule that limits false alarms → lowest threshold with validation FPR ≤ 5% chosen.
Evidence produced this week	• docs/PROJECT_KNOWLEDGE_MAP.md (requirement → module → test mapping) • docs/database.md (scan_history design) • ml/url_utils.py, ml/feature_extractor.py (started) • Phase 3 workbook pages 31–41
GUIDE FEEDBACK / COMMENTS	[Guide to complete after review]
Action plan for next week	• Phase 4: implement extractor and parity audit (O1) • Implement grouped split and train candidate models (O2) • Start Flask API skeleton
Guide Signature & Date	[Guide signs]
Team Lead Signature & Date	[Team lead signs] — team lead **[NEEDS USER INPUT]**

E. SUGGESTED POINTS TO RAISE WITH THE GUIDE (not guide feedback — do not write as feedback)

- Approval to move to Phase 4 implementation
- Is NFR-01 acceptable without a numeric target, or should one be set?

SECTION 16

Final Section — Missing Information / User Input Checklist

Tick each item before writing in the workbook.

[ ]	Item	Where needed	Why missing
[ ]	Mapping of the 4 names to Member 1–4 roles (ML, Backend, Frontend, Extension)	Pages 41, Week 6–8 logs	Docs give roles, PPT gives names, nothing links them
[ ]	Team lead name	Week 6–8 signature boxes	Not in repository
[ ]	Student IDs (if the workbook asks) — available on PPT slide 1	Cover / logs	Confirm they are correct
[ ]	Week 6, 7, 8 calendar dates (from – to)	Week logs, page 41	No dated log in repository
[ ]	Confirm O1/O2 are your CP1 objectives	Pages 31, 39	Repo does not label objectives "CP1"
[ ]	Confirm weekly activities and progress %	Week 6–8 logs	Reconstructed from documented work
[ ]	Review 1 date, slides and panel remarks	Week 6 log	No Review-1 file in repository
[ ]	Guide feedback / comments (each week)	Week 6–8 logs	Must come from the guide
[ ]	Guide signature & date; team lead signature & date	Week 6–8 logs	Handwritten
[ ]	NFR-01 numeric response-time target (optional)	Page 31	Never set in repository
[ ]	Whether PPT-only features (feedback table, automatic extension scanning, 12 endpoints) exist	Part A, diagrams	Not in committed code

Assumptions made in this guide

- Objectives are numbered O1–O6 in the order given on PPT slide 3 / review guide §1.
- Committed code (on-click extension, 9 endpoints, schema v2) is the current implementation; PPT-only features were left out of requirements and diagrams.
- Team has 4 members (docs and PPT agree), not 3.
- Trade-off scores and weights are engineering judgement, not measured values.
- Week-level allocation of tasks is a reconstruction; the repository has no weekly record.

Consistency check performed

Check	Result
Requirements ↔ objectives	Every FR/NFR traces to O1–O6; every objective has ≥ 1 requirement
Selected concept ↔ implementation	Concept C = URL-only Gradient Boosting + Flask API (models/model_metadata.json)
Architecture ↔ code	Every box maps to a file in backend/, services/, ml/, database/, templates/, static/, extension/
Data flow ↔ architecture	Same stages as docs/architecture.md request flow; no ensemble, no DistilBERT stage
Models named consistently	Gradient Boosting v2.0.0 everywhere; DistilBERT only as rejected Concept D
Dataset / metrics consistent	PhiUSIIL; 235,795 → 235,362; recall 0.5675, FPR 0.0554 — same figures in every section
Detailed design ↔ O1/O2	All 9 components belong to the ML pipeline that delivers O1 and O2
Weekly progression	W6 feasibility + requirements → W7 concepts + design → W8 finalised design + start of O1